CLOUD BASED EMPLOYEE ATTENDANCE SYSTEM

CLOUD COMPUTING

Submitted by

SOWBARNIGAA SRIDHARAN (230701327)

SOWMYA R (230701328)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

MAY 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled “CLOUD BASED EMPLOYEE ATTENDANCE SYSTEM" is the bonafide work of SOWBARNIGAA SRIDHARAN (230701327), SOWMYA R (230701328) who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E.M. Malathy

Head of the Department

Department Of Computer Science and Engineering

Rajalakshmi Engineering College

SIGNATURE

Dr. S.S. Senthil Kumar

Assistant Professor

Department of Computer Science and Engineering

Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on_____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman Mr. S.MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to Dr. S.S. Senthil Kumar, Assistant Professor from the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, DR. M. AYYADURAI M.E.,Ph.D., Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

SOWBARNIGAA SRIDHARAN

SOWMYA R

ABSTRACT

The Cloud-Based Attendance Management System is a full-stack web application designed to automate and streamline employee attendance and workforce management processes. Traditional attendance systems are often inefficient, difficult to manage, and lack centralized monitoring capabilities. This system addresses these challenges by providing a secure, scalable, and cloud-hosted platform for managing attendance, schedules, and employee activities in real time.

The application enables employees to securely log in, mark attendance, view assigned schedules, and manage leave requests, while administrators can monitor attendance records, assign shifts, manage employee details, and oversee workforce operations through a centralized dashboard. Role-based authentication and JWT-based authorization mechanisms ensure secure access control and protect sensitive employee data.

The system is developed using modern full-stack technologies, with the frontend built using React.js and Vite for an interactive user experience, and the backend implemented using Node.js and Express.js for handling business logic and RESTful API communication. SQLite is used as the database for storing employee, attendance, shift, and leave management data.

To achieve cloud accessibility and scalability, the project is deployed on Amazon Web Services (AWS) using an EC2 Ubuntu instance. The deployment includes frontend and backend hosting, process management using PM2, and secure server configuration through AWS Security Groups. GitHub is used for version control and deployment management.

The project demonstrates the practical implementation of cloud computing, full-stack development, authentication systems, RESTful APIs, and cloud deployment techniques to build a real-world workforce management solution. Overall, the system provides an efficient, centralized, and scalable approach to attendance management for organizational environments.

TABLE OF CONTENTS

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

Managing employee attendance and workforce operations manually is a major challenge for many organizations. Traditional attendance systems mainly depend on paper records, spreadsheets, or standalone applications, which are often inefficient, time-consuming, and difficult to maintain. These methods can lead to inaccurate attendance records, human errors, and difficulties in monitoring employee activities effectively.

One of the major issues in existing systems is the lack of centralized management. Administrators face difficulties in handling employee schedules, attendance records, leave requests, and shift assignments through separate or manual processes. This increases management complexity and reduces operational efficiency. In addition, manual attendance tracking does not provide real-time monitoring, making it difficult for organizations to analyze employee working hours and attendance status instantly.

Another challenge is limited accessibility. Traditional systems are usually accessible only within a local environment, restricting employees and administrators from accessing attendance information remotely. This becomes a major limitation in modern organizations where cloud-based accessibility and remote workforce management are important requirements.

Security is also a significant concern in many attendance systems. Basic systems often lack proper authentication and role-based access control mechanisms, increasing the risk of unauthorized access, data modification, and information leakage. Maintaining employee data securely while allowing controlled access is essential for organizational reliability.

Shift scheduling and leave management are additional operational challenges. Managing schedules manually for multiple employees can result in scheduling conflicts, incorrect shift allocation, and inefficient workforce planning. Similarly, handling leave requests manually consumes additional administrative effort and increases processing delays.

Furthermore, as organizational data grows, traditional attendance systems become difficult to scale and maintain. Manual record management reduces productivity and makes long-term data handling more complex. Organizations require a scalable and efficient solution capable of handling increasing employee data and operational requirements.

To overcome these limitations, a cloud-based attendance management system is required that provides centralized workforce management, secure authentication, real-time monitoring, and

remote accessibility. The proposed system aims to automate attendance tracking, shift scheduling, and leave management through a modern full-stack web application deployed on cloud infrastructure. By integrating cloud computing technologies and secure RESTful APIs, the system improves operational efficiency, accessibility, scalability, and overall workforce management.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of this project is to design and implement a cloud-based Attendance Management System that automates employee attendance tracking, workforce management, and schedule handling through a secure and centralized web platform.

1. Automate Attendance Management: Develop a system that enables employees to mark attendance digitally and allows administrators to monitor attendance records efficiently without manual intervention.

2. Provide Centralized Workforce Management: Create a unified platform for managing employee details, schedules, shift assignments, and leave requests through an organized administrative dashboard.

3. Enable Secure Authentication and Access Control: Implement JWT-based authentication and role-based authorization to ensure secure access for both administrators and employees.

4. Support Real-Time Monitoring: Allow administrators to track attendance status, employee activity, and workforce operations in real time for improved operational efficiency.

5. Build a Scalable Cloud-Based Platform: Deploy the application on AWS cloud infrastructure to provide remote accessibility, scalability, and reliable system performance.

6. Improve Operational Efficiency: Reduce manual paperwork, minimize human errors, and simplify workforce management through automation and centralized data handling.

7. Provide User-Friendly Accessibility: Develop an interactive and responsive web interface using modern full-stack technologies for easy access across different devices and environments.

1.3 SCOPE AND BOUNDARIES

The scope of this project includes the complete design, development, and cloud deployment of a full-stack Attendance Management System for managing employee attendance, schedules, and workforce operations through a centralized web application.

The system supports core functionalities such as employee and admin authentication, attendance tracking, shift scheduling, leave request management, and real-time workforce monitoring. Employees can securely log in, mark attendance, view schedules, and apply for leave, while administrators can manage employee records, monitor attendance data, assign schedules, and oversee organizational operations through an administrative dashboard.

The application is developed using React.js for the frontend and Node.js with Express.js for the backend. SQLite is used for data storage, and the entire system is deployed on AWS cloud infrastructure using an EC2 Ubuntu instance. RESTful APIs are implemented to enable communication between frontend and backend services.

The project focuses on providing a cloud-accessible and scalable workforce management solution suitable for organizations and institutional environments. The system is designed as a web-based application and currently does not include a dedicated mobile application. Advanced features such as biometric authentication, AI-based analytics, and payroll integration are outside the current scope of the project.

1.4 STAKEHOLDERS AND END USERS

Stakeholder: Project Developers / Team Members

Design, develop, deploy, and maintain the system modules such as authentication, attendance tracking, employee management, schedule handling, leave management, database integration, and cloud deployment services.

End Users:

1. Employees who use the system to securely log in, mark attendance, view schedules, and manage leave requests through the web application.

2. Administrators and HR Personnel who manage employee records, monitor attendance, assign work schedules, approve leave requests, and oversee workforce operations through the admin dashboard.

3. Organization Management seeking a centralized and cloud-based platform for efficient workforce monitoring, attendance management, and operational control in real time.

1.5 TECHNOLOGIES USED

The project incorporates a modern full-stack technology stack with cloud deployment on AWS, designed for secure attendance management, workforce monitoring, and scalable web application hosting.

Table 1. AWS Services

Component	Technology / Service	Purpose
Cloud Platform	AWS EC2	Host and deploy frontend and backend services
Frontend	React.js + Vite	Build interactive and responsive user interface
Backend	Node.js + Express.js	Handle server-side logic and RESTful APIs
Database	SQLite	Store employee, attendance, and schedule data
Authentication	JWT Authentication	Secure login and role-based access control
API Communication	REST APIs	Enable frontend-backend communication
Process Management	PM2	Manage backend server processes on AWS
Version Control	GitHub	Source code management and deployment workflow

Table 2. AI and Matching Technologies

Technology	Purpose
HTML, CSS, JavaScript	Frontend structure and styling
React Components	Build reusable UI modules
Express Middleware	Handle requests and authentication
AWS Security Groups	Configure secure server access
Ubuntu Server	AWS EC2 operating environment
Git & GitHub Actions	Code versioning and deployment automation

1.6 ORGANIZATION OF THE REPORT

This report is structured to provide a complete and systematic documentation of the development and deployment of the Cloud-Based Attendance Management System.

Chapter 1 introduces the project background, problem statement, objectives, scope, stakeholders, technologies used, and overall report organization.

Chapter 2 presents the system design and architecture, including functional requirements, proposed system overview, database structure, application workflow, and AWS cloud deployment architecture.

Chapter 3 explains the implementation details of the system, including frontend development, backend API integration, authentication mechanisms, attendance management modules, and database operations.

Chapter 4 focuses on cloud deployment and security aspects, including AWS EC2 deployment, server configuration, REST API communication, authentication, access control, and process management using PM2.

Chapter 5 presents the implementation results, system outputs, deployment outcomes, challenges faced during development and deployment, and performance observations.

Chapter 6 concludes the report with a summary of project achievements, overall system benefits, and future enhancement possibilities.

The report concludes with references and appendices containing screenshots, configuration details, database schemas, and deployment-related information.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

The proposed Attendance Management System is a cloud-based full-stack web application deployed on Amazon Web Services (AWS). The system is designed to automate employee attendance tracking, workforce management, shift scheduling, and leave management through a centralized platform.

The application consists of a frontend developed using React.js and Vite, which provides an interactive user interface for both employees and administrators. The backend is implemented using Node.js and Express.js, which handle business logic, authentication, RESTful API communication, and database operations.

Authentication functionality allows employees and administrators to securely log in to the system using JWT-based authentication and role-based access control. Employees can mark attendance, view assigned schedules, and submit leave requests, while administrators can manage employee records, assign shifts, monitor attendance data, and oversee workforce operations through an administrative dashboard.

Attendance data, employee details, schedules, and leave requests are stored in a SQLite database integrated with the backend server. REST APIs are used to enable secure communication between frontend and backend services.

The entire system is deployed on an AWS EC2 Ubuntu instance. PM2 is used for process management to ensure continuous backend execution, while GitHub is used for source code management and deployment updates.

Non-Functional Requirements

The system is designed to satisfy important non-functional requirements related to performance, scalability, security, reliability, and usability.

Performance: The application is designed to provide fast response times for attendance tracking, authentication, and schedule management operations with minimal delay during API communication.

Scalability: Deployment on AWS cloud infrastructure enables the system to support increasing numbers of employees and organizational data without significant performance degradation.

Availability: Hosting the application on AWS EC2 ensures remote accessibility and continuous system availability for both employees and administrators.

Security: JWT-based authentication and role-based authorization mechanisms are implemented to secure employee data and prevent unauthorized access to system resources.

Reliability: PM2 process management is used to maintain backend service availability and automatically restart the server in case of failures.

Usability: The React-based frontend provides a responsive and user-friendly interface that simplifies attendance management and workforce operations.

Maintainability: The modular full-stack architecture allows easier maintenance, debugging, and future feature enhancements.

Cost-Efficiency: The system uses lightweight technologies such as SQLite and AWS EC2 hosting to provide an affordable cloud-based solution suitable for small and medium-scale organizational environments.

2.2 PROPOSED SOLUTION OVERVIEW

The Cloud-Based Attendance Management System is designed as a full-stack web application that separates functionality across three major layers: the User Interface Layer, Backend Services Layer, and Database Layer.

At the presentation layer, a React.js-based web application provides an interactive interface for employees and administrators to access the system. Users authenticate securely through JWT-based login mechanisms and interact with the application to manage attendance, schedules, and leave requests.

The backend services layer is developed using Node.js and Express.js, which handle business logic, authentication, RESTful API communication, attendance processing, employee management, shift scheduling, and leave management functionalities. The backend processes user requests and securely communicates with the database layer.

The database layer uses SQLite to store employee records, attendance details, schedules, shifts, and leave request information. The database is integrated with the backend server to support efficient data storage and retrieval operations.

The entire system is deployed on AWS EC2 cloud infrastructure using an Ubuntu server environment. PM2 is used for backend process management to ensure continuous application availability and automatic restart support. GitHub is used for source code management and deployment updates.

The proposed solution provides a centralized, secure, and cloud-accessible workforce management platform that improves attendance tracking efficiency, reduces manual administrative work, and enables real-time monitoring of organizational operations.

2.3 CLOUD DEPLOYMENT STRATEGY

The deployment strategy for the proposed Attendance Management System utilizes Amazon Web Services (AWS) as the cloud platform for hosting and managing the application. The system follows a cloud-based deployment model to provide remote accessibility, centralized management, scalability, and reliable application performance.

The frontend and backend applications are deployed on an AWS EC2 Ubuntu instance. The React.js frontend is hosted to provide users with a responsive web interface, while the Node.js and Express.js backend handles RESTful APIs, authentication, attendance processing, and workforce management functionalities.

SQLite is used as the database for storing employee records, attendance details, schedules, shifts, and leave management information. The database is integrated directly with the backend server running on the EC2 instance.

PM2 process management is used to maintain continuous backend execution and automatically restart the application in case of runtime failures. AWS Security Groups are configured to securely manage network access and allow controlled communication through required application ports.

GitHub is used for source code management and deployment updates. Application files are deployed and maintained on the AWS server using Git-based workflows and remote server configuration.

The overall system flow is: User accesses the web application through the frontend → User requests are sent to the Node.js backend via REST APIs → Backend processes attendance, authentication, schedule, and leave operations → SQLite database stores and retrieves system data → Results are returned to the frontend for real-time user interaction.

The cloud deployment strategy provides centralized application hosting, improved accessibility, scalability, and efficient workforce management through AWS cloud infrastructure.

2.4 INFRASTRUCTURE REQUIREMENTS

Compute Resources:

• AWS EC2 Instance: Ubuntu-based EC2 virtual machine used to host both frontend and backend applications. The instance runs the React.js frontend and Node.js backend services continuously.

• Node.js Runtime Environment: Used to execute backend server operations, RESTful APIs, authentication modules, attendance processing, and database communication.

• PM2 Process Manager: Used for backend process management, automatic restart handling, and maintaining continuous server availability.

Storage Requirements:

• SQLite Database: Used for storing employee records, attendance details, schedules, shifts, authentication data, and leave management information.

• AWS EC2 Storage: Local server storage attached to the EC2 instance for hosting application files, database files, and deployment resources.

• GitHub Repository: Used for source code storage, version control, and deployment management of frontend and backend project files.

Network and Security Requirements:

• AWS Security Groups: Configured to allow secure inbound and outbound traffic for application access through required ports such as HTTP, HTTPS, and backend service ports.

• JWT Authentication: Implemented for secure user authentication and role-based access control between administrators and employees.

• REST API Communication: Enables secure frontend-backend interaction and data exchange across the cloud-hosted application environment

2.5 AWS SERVICES MAPPING AND JUSTIFICATION

Table 3. AWS Service Mapping

Requirement	AWS Service / Technology	Purpose	Justification
Cloud Hosting	AWS EC2	Host frontend and backend applications	Provides scalable and centralized cloud hosting environment
Operating System	Ubuntu Server	Run application services on EC2	Stable and widely used Linux server environment
Frontend Development	React.js + Vite	Build responsive user interface	Enables fast and interactive web application development
Backend Services	Node.js + Express.js	Handle business logic and REST APIs	Lightweight and efficient backend framework
Database	SQLite	Store attendance and employee data	Simple and lightweight database suitable for project-scale deployment
Authentication	JWT Authentication	Secure login and authorization	Provides secure session handling and role-based access control
Process Management	PM2	Maintain backend server processes	Ensures continuous backend availability and automatic restart support
Version Control	GitHub	Manage source code and deployment	Enables collaborative development and deployment management
Network Security	AWS Security Groups	Control server access and traffic	Provides secure inbound and outbound communication
API Communication	REST APIs	Connect frontend and backend	Enables secure and efficient data exchange between application layers

CHAPTER 3 - SYSTEM IMPLEMENTATION

3.1 SYSTEM IMPLEMENTATION OVERVIEW

The Cloud-Based Attendance Management System is designed as a full-stack web application that automates employee attendance tracking, schedule management, and workforce operations through a centralized cloud-hosted platform. The system integrates frontend, backend, database, authentication, and cloud deployment components to provide secure and efficient workforce management.

The application follows a role-based architecture where administrators and employees have different levels of access and functionalities. Employees can securely log in, mark attendance, view assigned schedules, and submit leave requests, while administrators can manage employee details, monitor attendance records, assign shifts, and oversee workforce activities through an administrative dashboard.

The frontend is developed using React.js and Vite, providing a responsive and interactive user interface for system users. The backend is implemented using Node.js and Express.js, which handle RESTful API communication, authentication, business logic, and database operations. SQLite is used as the database for storing employee information, attendance data, schedules, and leave records.

JWT-based authentication is implemented to secure user sessions and enforce role-based access control. REST APIs enable communication between frontend and backend modules for real-time data exchange and application functionality.

The entire system is deployed on AWS EC2 cloud infrastructure using an Ubuntu server environment. PM2 process management is used to maintain backend service availability and automatically restart backend processes during failures or server restarts.

The system aims to improve organizational efficiency by reducing manual attendance management efforts, centralizing workforce operations, and enabling cloud-based accessibility for administrators and employees.

3.2 FRONTEND AND BACKEND IMPLEMENTATION

Frontend Implementation using React.js:

The frontend of the Attendance Management System is developed using React.js and Vite to provide a responsive and user-friendly web interface. The application is designed with multiple modules that allow employees and administrators to interact with the system efficiently.

The frontend includes login authentication pages, attendance management dashboards, employee management modules, schedule viewing interfaces, and leave request forms. React components are used to build reusable UI elements and manage dynamic content updates within the application.

Axios is used for API communication between the frontend and backend services. User authentication tokens are stored securely in local storage to maintain user sessions and enable protected route access within the application.

Backend Implementation using Node.js and Express.js:

The backend is implemented using Node.js and Express.js to handle server-side operations, RESTful APIs, authentication, attendance processing, and workforce management functionalities.

The backend server processes user requests related to attendance tracking, employee management, shift scheduling, and leave management. Role-based access control is implemented using JWT authentication to ensure secure access for administrators and employees.

Express middleware is used for request handling, authentication verification, and API routing. The backend also manages database operations and communicates with the SQLite database to store and retrieve attendance and employee-related data.

Database Integration using SQLite:

SQLite is used as the database system for storing employee details, attendance records, schedules, shifts, and leave request information. The database is integrated directly with the backend server to support efficient data storage and retrieval operations.

The database schema includes tables for employees, attendance, schedules, shifts, and leave management. SQL queries are executed through backend APIs to perform create, update, delete, and retrieval operations for workforce management activities.

Combined System Workflow:

The complete system workflow begins when a user accesses the React-based frontend application. User requests are sent to the Node.js backend through REST APIs. The backend processes authentication, attendance operations, and schedule management tasks before interacting with the SQLite database. The processed results are then returned to the frontend for real-time display and user interaction.

3.3 DATABASE MANAGEMENT AND API COMMUNICATION

The Attendance Management System uses SQLite as the primary database for storing employee information, attendance records, schedules, shifts, and leave request details. The database is integrated with the Node.js backend to support efficient data management and retrieval operations.

The database contains multiple tables for handling different system functionalities, including employee management, attendance tracking, shift allocation, scheduling, and leave management. SQL queries are executed through backend APIs to perform insert, update, delete, and retrieval operations based on user requests.

RESTful APIs are implemented using Express.js to establish communication between the frontend and backend layers. These APIs handle authentication, attendance marking, employee management, schedule assignment, and leave processing operations securely and efficiently.

JWT-based authentication is integrated into API communication to ensure secure access control and protect sensitive employee data. Only authorized users are allowed to access protected routes and perform specific operations based on their assigned roles.

The backend server processes incoming API requests, interacts with the SQLite database, and sends appropriate responses back to the React frontend. This centralized communication workflow enables real-time workforce management and smooth system functionality across the cloud-hosted platform.

3.4 SYSTEM WORKFLOW AND REQUEST PROCESSING

The Attendance Management System follows a structured workflow that automates attendance tracking, employee management, and workforce operations through cloud-based request processing and API communication.

The system workflow is as follows:

1. A user accesses the web application through the React.js frontend interface and logs in using secure authentication credentials.
2. The login request is sent to the Node.js backend through REST APIs, where JWT-based authentication validates the user and provides role-based access.
3. Employees can mark attendance, view schedules, and submit leave requests through the frontend dashboard, while administrators can manage employee records, assign shifts, and monitor attendance information.
4. The frontend sends user requests to the backend APIs using Axios-based HTTP communication.
5. The Express.js backend processes the requests, performs validation checks, and interacts with the SQLite database to store or retrieve required data.
6. Attendance records, employee details, schedules, shifts, and leave requests are stored and managed within the database through backend query operations.
7. The backend sends processed responses back to the frontend, where updated information is displayed to users in real time.
8. The entire application is hosted on AWS EC2 cloud infrastructure, allowing centralized access and remote workforce management capabilities.
9. PM2 process management continuously monitors the backend server and automatically restarts backend services in case of failures to maintain system availability and reliability.

3.5 AUTHENTICATION AND SECURITY MANAGEMENT

The Attendance Management System implements secure authentication and authorization mechanisms to protect employee data and ensure controlled system access. The application uses JWT (JSON Web Token)-based authentication to manage user sessions securely and maintain protected communication between frontend and backend services.

When a user logs into the system, the backend validates the login credentials and generates a JWT token. This token is stored securely on the client side and is included in subsequent API requests to access protected functionalities within the application. The authentication mechanism ensures that only verified users can access attendance management features and organizational data.

The system also implements role-based access control to differentiate permissions between employees and administrators. Employees can mark attendance, view schedules, and submit leave requests, while administrators are provided with additional privileges such as employee management, attendance monitoring, shift assignment, and workforce administration.

Express middleware is used in the backend to verify JWT tokens before processing protected API requests. Unauthorized users attempting to access restricted resources are denied access, improving overall system security and data protection.

The application is deployed on AWS EC2 cloud infrastructure, where AWS Security Groups are configured to control network traffic and allow only required communication ports. This helps secure the cloud-hosted environment from unauthorized external access.

PM2 process management is used to maintain backend service availability and automatically restart the Node.js server in case of failures or unexpected shutdowns. This improves application reliability and ensures continuous workforce management operations.

Overall, the implemented authentication and security mechanisms provide secure user access, protected API communication, controlled administrative operations, and reliable cloud-based system management.

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

4.1 AUTHENTICATION AND ACCESS CONTROL

Security is implemented at multiple levels within the Attendance Management System to ensure secure user access, protected data handling, and reliable cloud operations.

User authentication is managed using JWT-based authentication. Employees and administrators must log in with valid credentials before accessing the system. After successful login, the backend generates a JWT token, which is used to authenticate subsequent API requests.

Role-based access control is implemented to restrict functionalities based on user roles. Employees can access attendance, schedules, and leave modules, while administrators have additional access to employee management, attendance monitoring, and schedule assignment features.

The backend uses Express middleware to validate JWT tokens before processing protected API requests. Unauthorized requests with missing or invalid tokens are rejected to maintain application security.

The application is deployed on AWS EC2 cloud infrastructure, where AWS Security Groups are configured to control inbound and outbound traffic. Only required ports are enabled to provide secure network communication between users and the cloud-hosted application.

Sensitive data such as authentication credentials and environment configurations are managed using environment variables within the backend server, reducing the risk of exposing confidential information in the application codebase.

4.2 MONITORING AND OBSERVABILITY

The Attendance Management System uses monitoring and management mechanisms to ensure reliable application performance and continuous cloud operation on AWS infrastructure.

The application deployed on AWS EC2 is monitored through server logs, backend console outputs, and process management tools. PM2 is used to continuously monitor the Node.js backend server and automatically restart the application if any runtime failure or unexpected shutdown occurs.

Backend API requests, authentication operations, attendance processing, and database interactions are monitored through server-side logging mechanisms implemented within the Express.js backend. These logs help identify errors, API failures, and operational issues during system execution.

System performance is also observed through AWS EC2 instance monitoring, including server availability, CPU usage, memory usage, and network activity. AWS Security Groups further help monitor and control secure network communication to the application server.

The frontend and backend integration is tested continuously to ensure smooth API communication and proper workforce management functionality. Monitoring these operations helps maintain system stability, improve reliability, and quickly identify technical issues during deployment and usage.

Overall, the monitoring and observability mechanisms improve system reliability, backend availability, cloud performance management, and operational stability for the Attendance Management System.

4.3 CLOUD OPERATIONS MANAGEMENT

The Attendance Management System follows a cloud-based operational model using AWS EC2 infrastructure for hosting frontend and backend services. Since the application is deployed on a centralized cloud server, operational activities mainly focus on application management, server monitoring, and service availability.

The Node.js backend server is managed using PM2, which ensures continuous application execution and automatically restarts backend processes during failures or unexpected shutdowns. This improves application reliability and minimizes downtime during system operation.

Application dependencies and project packages are managed using Node Package Manager (NPM). Backend and frontend modules are maintained separately, allowing easier updates and deployment management without affecting the overall system functionality.

GitHub is used for source code version control and deployment updates. Changes made to the frontend or backend codebase can be deployed and maintained efficiently through Git-based workflows.

AWS EC2 provides centralized cloud hosting, allowing remote access to the application from different environments. AWS Security Groups are configured to securely manage server communication and application access through required network ports.

Overall, the cloud operations management approach improves application reliability, deployment flexibility, backend availability, and centralized workforce management through AWS cloud infrastructure.

4.4 DISASTER RECOVERY PLANNING

The Attendance Management System includes basic disaster recovery and backup strategies to ensure application reliability and data availability within the AWS cloud environment.

The application source code is maintained using GitHub version control, allowing quick recovery and redeployment of frontend and backend services in case of accidental data loss, configuration issues, or server failures. All project files, updates, and deployment changes are securely stored within the Git repository.

The system is hosted on an AWS EC2 instance, which provides reliable cloud infrastructure and continuous application availability. In case of backend failure or unexpected shutdown, PM2 process management automatically restarts the Node.js server to minimize downtime and maintain uninterrupted service operation.

SQLite database files and application resources can be backed up periodically to prevent data loss and support recovery during server issues or deployment failures. Since the application is cloud-hosted, redeployment and restoration can be performed quickly using the stored project repository and server configurations.

AWS Security Groups and controlled access settings also help maintain secure application recovery and prevent unauthorized modifications during operational failures.

The disaster recovery approach focuses on minimizing service interruption, restoring backend operations quickly, and maintaining reliable workforce management functionality through cloud-based deployment and backup strategies.

APPENDIX C - PROJECT OUTPUT SCREENSHOTS

The following screenshots demonstrate the complete working implementation of the Cloud Based Employee Attendance System.

1. User Registration

The secure login interface for employees and administrators using JWT-based authentication. Users can access attendance and workforce management features based on their assigned roles.

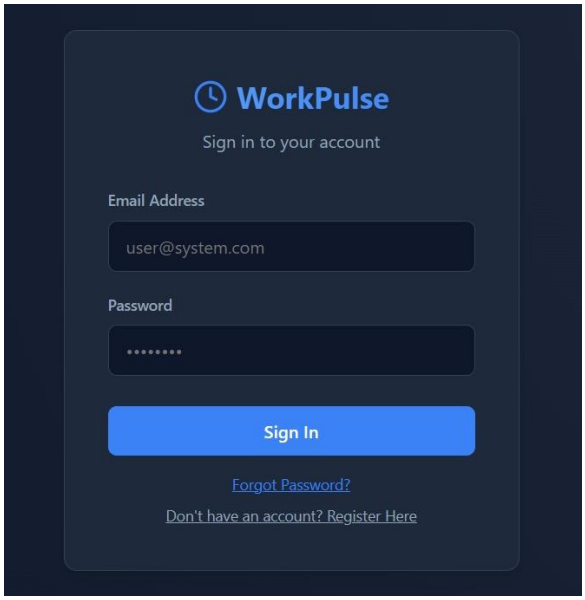

Figure 1. WorkPulse - Create Account Page

2. Reports Page

The analytical dashboard displaying present, absent, and late attendance statistics through graphical bar chart visualization for effective workforce monitoring and analysis.

Figure 2. WorkPulse - Reports

3. Admin Dashboard

The centralized administrative dashboard that provides access to employee reports, leave requests, attendance records, schedules, and profile management through an organized navigation panel.

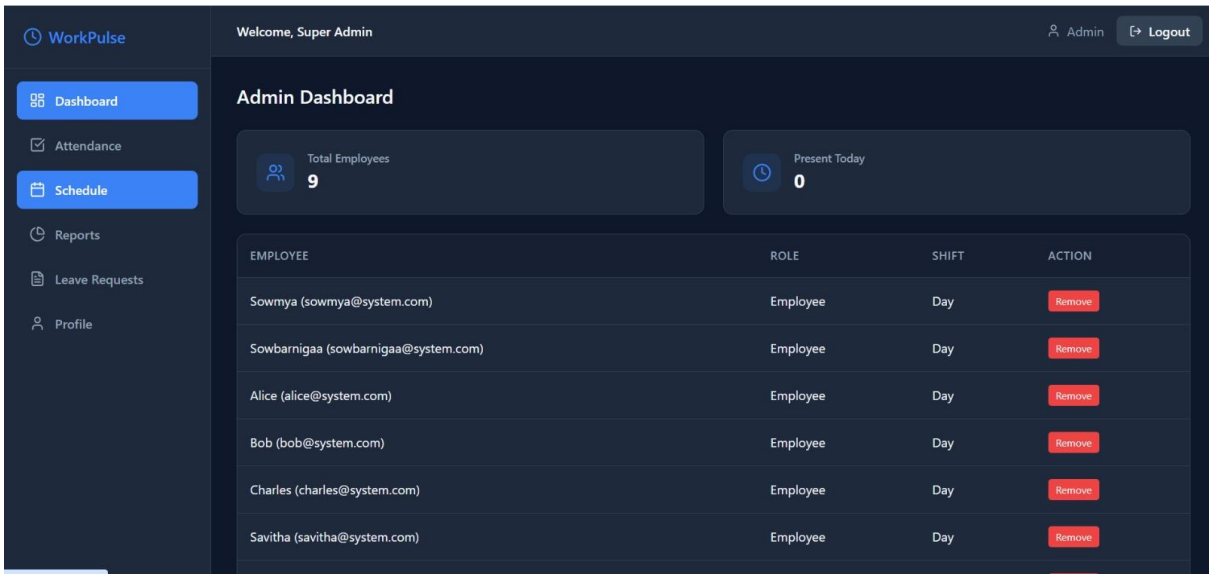

Figure 3. WorkPulse – Admin Dashboard

4. Leave Request Page

The interface that allows employees to submit leave requests and enables administrators to manage and review leave applications efficiently.

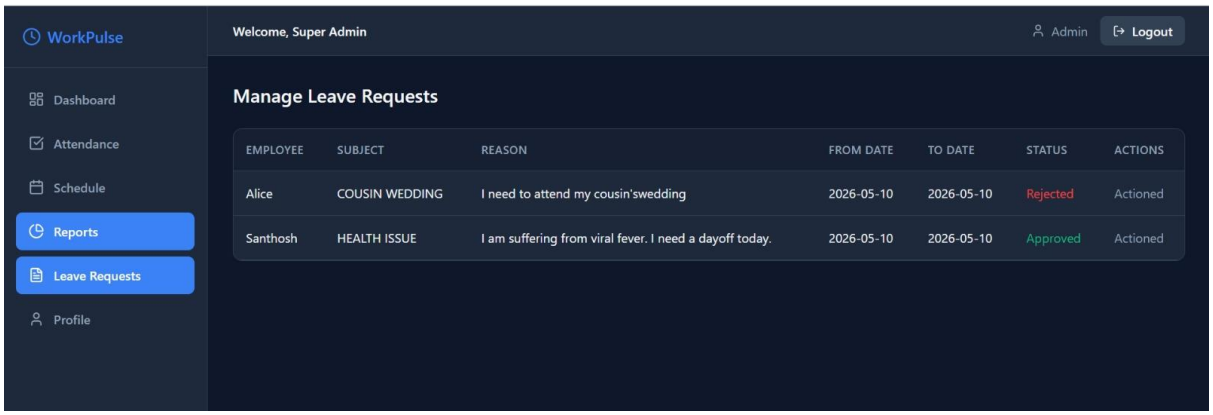

Figure 4. WorkPulse – Leave Request Page

5. Employee Dashboard

The attendance management interface that allows employees to mark daily check-in and check-out times and maintain real-time attendance records.

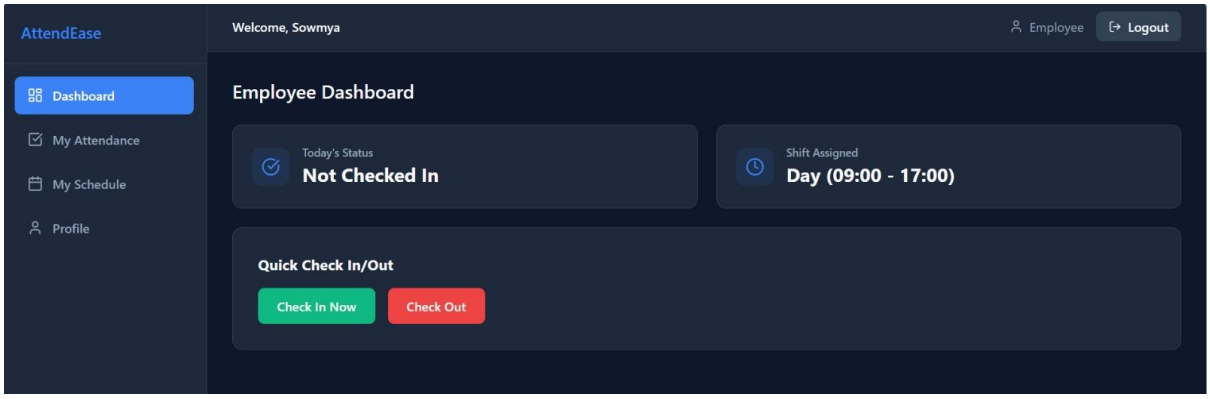

Figure 5. WorkPulse – Employee Dashboard

6. Assign Schedules Page

The administrative interface used to assign work schedules and shifts to employees for effective workforce planning and management.

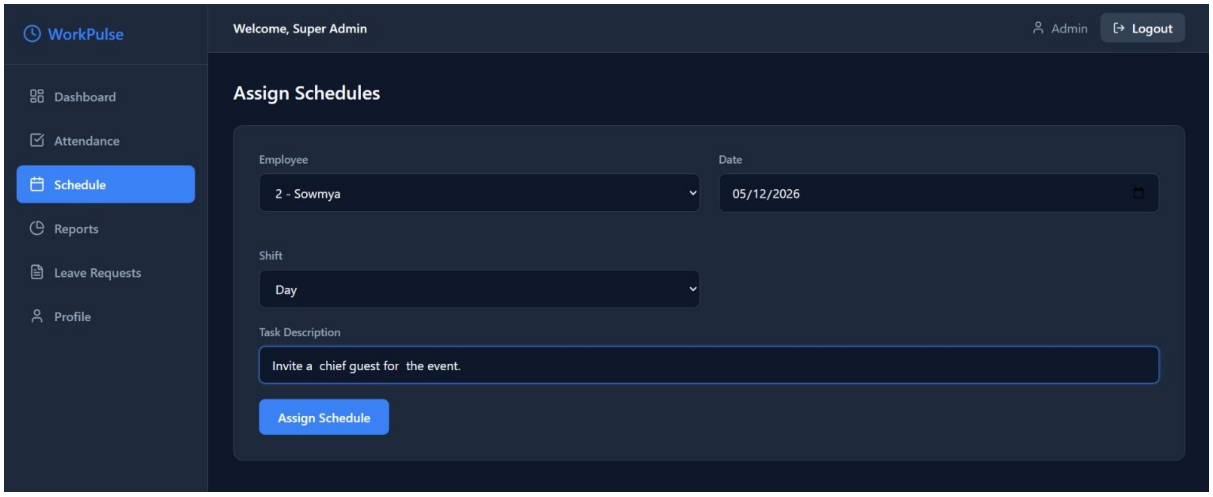

Figure 6. WorkPulse – Assign Schedules Page

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The Cloud-Based Attendance Management System was successfully developed and deployed on Amazon Web Services (AWS), achieving the primary objectives defined in the project scope. The system integrates attendance tracking, employee management, schedule assignment, leave management, authentication, and cloud deployment within a centralized full-stack web application.

The application was implemented using React.js for the frontend, Node.js and Express.js for the backend, and SQLite for database management. JWT-based authentication and role-based access control were successfully integrated to provide secure access for employees and administrators.

The deployed system allows employees to securely log in, mark attendance, view schedules, and submit leave requests, while administrators can manage employee records, assign schedules, monitor attendance reports, and oversee workforce operations through the admin dashboard.

The frontend and backend services were successfully hosted on AWS EC2 cloud infrastructure using an Ubuntu server environment. PM2 process management ensured continuous backend availability and automatic restart support during runtime failures.

RESTful APIs enabled smooth communication between frontend and backend modules, supporting real-time workforce management operations and centralized data handling. Attendance reports and graphical visualizations provided effective monitoring of present, absent, and late employee statistics.

The cloud deployment demonstrated improved accessibility, centralized workforce management, operational efficiency, and reliable application performance. The project successfully showcased the practical implementation of full-stack development, cloud computing, authentication systems, and cloud-hosted workforce management solutions.

5.2 CHALLENGES FACED AND RESOLUTIONS

Backend Deployment Issues:

During AWS deployment, the Node.js backend initially failed to run due to server configuration and dependency-related issues. This was resolved by properly configuring the

EC2 Ubuntu environment, installing required Node.js packages, and managing backend services using PM2.

Database Connection Problems:

The SQLite database faced file access and connection issues during cloud deployment. The problem was resolved by correcting database file paths and configuring proper backend access permissions within the AWS server environment.

Frontend and Backend Integration:

API communication between the React frontend and Node.js backend initially failed due to incorrect localhost API references after deployment. This issue was resolved by replacing local API URLs with deployed backend endpoints and configuring proper REST API communication.

Authentication and Access Control Issues:

JWT authentication initially caused token validation and protected route access issues. Middleware-based token verification and role-based access control were implemented to ensure secure user authentication and authorization.

Cloud Deployment and Process Management:

Application downtime occurred during backend execution failures. This issue was resolved using PM2 process management, which automatically restarts backend services and maintains continuous application availability on the AWS EC2 server.

5.3 PERFORMANCE OR COST OBSERVATIONS

Performance Benchmarking:

The Attendance Management System demonstrated stable and reliable performance during testing and cloud deployment. Frontend pages developed using React.js provided smooth navigation and responsive user interaction for attendance tracking, schedule management, and leave processing operations.

REST API communication between the frontend and backend showed fast response times for authentication, attendance updates, employee management, and database retrieval operations. SQLite database queries executed efficiently for handling employee records, attendance details, and scheduling information.

The Node.js backend hosted on AWS EC2 maintained continuous service availability through PM2 process management, which improved backend reliability and minimized downtime during runtime failures.

Cost Observations:

The system was deployed using AWS EC2 cloud infrastructure with lightweight technologies such as Node.js, React.js, and SQLite, making the project cost-effective for small and medium-scale organizational environments.

Since SQLite is a lightweight database solution and the application is hosted on a single AWS EC2 instance, infrastructure and operational costs remained relatively low. GitHub was used for free source code management and deployment support, further reducing overall development expenses.

The cloud deployment approach demonstrated that a centralized attendance management platform can be implemented efficiently with minimal infrastructure cost while still providing secure access, scalability, and reliable workforce management functionality.

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

Throughout the development of the Cloud-Based Attendance Management System, the project provided valuable practical experience in full-stack development, cloud deployment, authentication systems, and workforce management application design.

A major technical learning was understanding cloud deployment using AWS EC2 infrastructure. The project helped in learning server configuration, backend hosting, API deployment, security group management, and process management using PM2 for maintaining continuous backend availability.

The development process also improved knowledge of frontend and backend integration using React.js, Node.js, and Express.js. Implementing RESTful APIs, JWT authentication, role-based access control, and database management using SQLite provided practical experience in secure web application development.

The team also gained experience in debugging deployment issues, managing cloud-hosted applications, handling API communication errors, and maintaining centralized application workflows in a cloud environment.

From a teamwork perspective, responsibilities were divided across frontend development, backend implementation, database integration, cloud deployment, authentication handling, testing, and documentation. All team members contributed collaboratively to system development, troubleshooting, deployment, report preparation, and final project integration.

The project significantly enhanced technical knowledge in cloud computing, web application development, database management, authentication systems, and real-world deployment practices using AWS cloud services.

CHAPTER 6 - CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

This project successfully developed and deployed a Cloud-Based Attendance Management System on Amazon Web Services (AWS) to automate employee attendance tracking, workforce management, schedule assignment, and leave management operations. The system addressed the limitations of traditional manual attendance methods by providing a centralized, secure, and cloud-accessible web application.

The project demonstrated the successful integration of React.js frontend development, Node.js and Express.js backend implementation, SQLite database management, JWT-based authentication, and AWS EC2 cloud deployment. The application provided role-based access control for employees and administrators, enabling secure and efficient workforce management.

The cloud deployment on AWS EC2 proved reliable and cost-effective for hosting full-stack applications. PM2 process management improved backend availability and ensured continuous service operation during runtime failures. RESTful API communication enabled smooth interaction between frontend and backend modules, supporting real-time attendance and schedule management functionalities.

The system provided significant operational benefits by reducing manual paperwork, improving attendance monitoring efficiency, centralizing employee management, and enabling remote accessibility through cloud infrastructure. Attendance reporting and workforce monitoring features further enhanced organizational management capabilities.

The project also provided valuable practical experience in full-stack web development, cloud computing, deployment management, authentication systems, REST API integration, and real-world application hosting using AWS services.

Overall, the project successfully demonstrated the implementation of a secure, scalable, and cloud-based workforce management solution capable of supporting modern organizational attendance operations efficiently.

6.2 FUTURE WORKS

The current implementation of the Attendance Management System provides a strong foundation for several future enhancements and advanced workforce management features.

Mobile Application Support:

A dedicated mobile application for Android and iOS platforms can be developed to allow employees and administrators to access attendance and workforce management features more conveniently through smartphones.

Biometric Attendance Integration:

Future versions of the system can integrate biometric technologies such as face recognition or fingerprint authentication to improve attendance accuracy and reduce proxy attendance issues.

Advanced Reporting and Analytics:

Additional analytical dashboards and graphical reports can be implemented to provide detailed insights into employee attendance trends, productivity, and workforce performance.

Notification and Email Integration:

Automated email or SMS notifications can be added for attendance alerts, leave approvals, schedule updates, and important workforce announcements.

Payroll Integration:

Future enhancements may include payroll management features where employee salaries can be calculated automatically based on attendance records, working hours, and leave information.

Multi-Organization Support:

The system can be extended into a multi-tenant platform capable of supporting multiple organizations or departments within a single centralized cloud environment.

These future improvements can enhance scalability, usability, automation, and overall workforce management efficiency within the cloud-based attendance management platform.

REFERENCES

1. Amazon Web Services. (2026). Amazon EC2 Documentation. https://docs.aws.amazon.com/ec2/

2. Amazon Web Services. (2026). AWS Security Groups Documentation. https://docs.aws.amazon.com/vpc/

3. Node.js Foundation. (2026). Node.js Documentation. https://nodejs.org/

4. Express.js. (2026). Express.js Web Framework Documentation. https://expressjs.com/

5. React Documentation Team. (2026). React.js Documentation. https://react.dev/

6. SQLite Documentation. (2026). SQLite Official Documentation. https://www.sqlite.org/docs.html

7. JWT.io. (2026). JSON Web Token Introduction. https://jwt.io/introduction

8. PM2 Documentation. (2026). PM2 Process Manager Documentation. https://pm2.keymetrics.io/

9. GitHub Documentation Team. (2026). GitHub Documentation. https://docs.github.com/

10. Axios Documentation. (2026). Axios HTTP Client Documentation. https://axios-http.com/docs/intro